

DEVHEAL LABS AI

DH-ADOM

DevHeal Hierarchical Agent Development & Orchestration Model

White paper 1.1 | 7 October 2026

Sai Narender Nuckala / DevHeal Labs AI

An engineering operating model for repository ownership, bounded delegation and evidence-backed handoffs.

Canonical publication: <https://devheallabs.com/dh-adom/>

Edition 1.1 · 7 October 2026. Editorial revision: generic adoption examples, canonical .com links, readable vector figures and refreshed layout. The model and normative specification remain version 1.0. No new empirical results are asserted.

Publication Note

This white paper is intended as the canonical public description of DH-ADOM. The model is positioned as an engineering operating model for AI-native repositories: it governs how AI development work is organized, delegated, bounded, validated, traced, and handed off across a hierarchy of agents.

The paper deliberately distinguishes DH-ADOM from adjacent technologies. Multi-agent frameworks such as AutoGen provide mechanisms for composing agents; orchestrator-worker patterns describe useful execution topologies; agent-development lifecycle guidance addresses agent-specific development; AI-SDLC initiatives target governance and orchestration of AI coding agents; and NIST, ISO, OWASP, MCP, and A2A address risk, management, security, or interoperability. DH-ADOM is proposed as a higher-level operating model that organizes repository ownership and delegation across Root, Feature, and Specialist agents while tying that hierarchy to AI-SDLC and ADLC.

Because the surrounding field is moving quickly, future DH-ADOM releases should maintain a versioned prior-art review and update this paper as standards, protocols, and research evolve.

Recommended Citation

Nuckala, S. N. / DevHeal Labs AI. (2026). DH-ADOM: DevHeal Hierarchical Agent Development & Orchestration Model. White Paper v1.1.

Abstract

AI coding systems are shifting software development from human-authored code toward delegated execution in repositories. Modern agents can inspect codebases, plan changes, invoke tools, create artifacts, run tests, coordinate other agents, and operate over long horizons. This creates a new engineering problem: how should an AI-native repository organize ownership, delegation, autonomy, evidence, and lifecycle control when multiple agents participate in the same product?

DH-ADOM (DevHeal Hierarchical Agent Development & Orchestration Model) proposes a repository-level operating model built around a governed hierarchy: an Engineering Constitution establishes the non-negotiable rules; AI-SDLC governs the software lifecycle; ADLC governs agent lifecycle; a Root/Main Orchestrator Agent manages project-level planning and delegation; Feature/Domain Agents own bounded feature areas; and Specialist Agents perform narrowly scoped tasks under explicit delegation contracts.

The model treats agent hierarchy as an engineering control boundary rather than merely a runtime topology. Ownership, permissions, autonomy, delegation depth, budgets, lifetime, handoff, traceability, evidence, and cross-feature changes become explicit and auditable. DH-ADOM therefore seeks to reduce agent sprawl, prevent the Root Agent from becoming a monolithic super-agent, preserve feature ownership, and make AI-resumable development possible without hidden chat memory.

This white paper defines the model, its formal control primitives, implementation blueprint, security considerations, evaluation methodology, limitations, and relationship to adjacent work. The intended result is not another agent framework, but a repeatable operating model for building and evolving agentic software systems under explicit engineering governance.

Keywords

AI-SDLC; ADLC; multi-agent systems; agent orchestration; software engineering agents; autonomous coding; feature agents; specialist agents; delegation governance; agent identity; agent contracts; repository governance; AI-resumable software.

1. Executive Summary

Traditional SDLC assumes that the primary implementer is a human engineer operating inside a stable team and organizational structure. AI-native development changes the execution layer: an agent may independently inspect a repository, select tools, create subplans, execute commands, modify source code, run validation, and delegate work to additional agents. The repository therefore becomes an environment containing both software artifacts and machine-executable organizational rules.

DH-ADOM addresses the organizational problem created by this shift. Its central proposition is that AI-native software engineering should use explicit hierarchical ownership rather than an undifferentiated pool of agents.

Layer	Primary responsibility	Control question
Engineering Constitution	Global rules and precedence	What may never be bypassed?
AI-SDLC	Software lifecycle governance	What lifecycle evidence is required?
ADLC	Agent lifecycle governance	Is this agent fit to exist and act?
Root Agent	Project-level orchestration	Which domain should do the work?
Feature Agent	Bounded domain ownership	What does this feature own?
Specialist Agent	Narrow technical assistance	What specific problem is delegated?
Evidence / Traceability	Proof of work and lineage	Can the work be reproduced and audited?

Core thesis

The unit of organization in an AI-native repository should not be the model or prompt. It should be the governed engineering responsibility.

See Figure A1 in Appendix A for the vector diagram.

ENGINEERING CONSTITUTION -> AI-SDLC -> ADLC -> ROOT / MAIN AGENT -> FEATURE / DOMAIN AGENTS -> BOUNDED SPECIALISTS -> IMPLEMENTATION + VALIDATION -> EVIDENCE + HANDOFF -> ROOT REVIEW + INTEGRATION -> CONTINUOUS AI-SDLC FEEDBACK

2. Problem Statement

Multi-agent frameworks make it increasingly easy to create systems where multiple agents collaborate, route work, or operate in orchestrator-worker patterns. Anthropic describes orchestrator-workers as a pattern in which a central model dynamically breaks down tasks, delegates them to workers, and synthesizes results. Microsoft's AutoGen provides a general framework for composing multiple agents and conversation patterns. These patterns are useful, but they do not by themselves answer the repository-level engineering questions of who owns a feature, who may change its files, how a specialist is created, what delegation authority exists, how long a delegated agent may live, how work is handed back, or how the hierarchy maps to software lifecycle evidence. [1][2]

At the same time, emerging AI-SDLC frameworks are beginning to formalize autonomous coding-agent orchestration, quality gates, declarative governance, and repository instructions. Salesforce describes an Agent Development Lifecycle as a dedicated lifecycle for autonomous agents, while emerging AI-SDLC projects describe orchestrators, agent roles, quality gates, autonomy policies, and auditability. [3][4]

The resulting gap is not the absence of orchestration primitives. The gap is a coherent operating model for hierarchical engineering responsibility inside an AI-native repository.

2.1 Engineering failure modes addressed

- Root-agent overreach: one central agent becomes responsible for every domain and every implementation detail.
- Feature ownership ambiguity: multiple agents can modify the same domain without an explicit owner.
- Unbounded delegation: agents recursively create agents without defined depth, cost, permissions, or lifetime.
- Agent sprawl: agents accumulate without lifecycle or registry discipline.
- Context fragmentation: new agents depend on hidden chat history rather than repository state.
- Cross-domain drift: one feature silently modifies another feature's implementation.
- Untraceable execution: the repository cannot reconstruct why an agent acted or which requirement caused a change.

- Governance bypass: agent execution proceeds despite missing requirements, contracts, tests, approvals, or evidence.

3. Research Landscape and Related Work

DH-ADOM should be read in the context of an active and rapidly changing research and engineering ecosystem. The model does not claim ownership of every underlying primitive. Instead, it composes and formalizes several known directions into a specific repository operating model.

Adjacent work	Primary contribution	Relationship to DH-ADOM
Anthropic agent patterns	Workflows, routing, parallelization, orchestrator-workers, evaluator-optimizer, agents	Supports the execution-topology foundation; DH-ADOM adds explicit engineering ownership and delegation governance.
Microsoft AutoGen	Multi-agent conversation and programmable agent collaboration	Provides agent-composition mechanisms; DH-ADOM defines repository-level hierarchy and ownership above runtime conversations.
AI-SDLC Framework	Declarative lifecycle governance, autonomous orchestration, quality gates, autonomy, audit	Closest neighboring governance work; DH-ADOM focuses specifically on Root -> Feature -> Specialist ownership and bounded domain delegation.
Salesforce ADLC	Agent lifecycle from conception through continuous monitoring	Supports the need for agent-specific lifecycle governance; DH-ADOM embeds ADLC inside a hierarchical development operating model.
NIST AI RMF / GAI Profile	Risk management, trustworthy AI lifecycle practices	Provides governance and risk-management context; DH-ADOM operationalizes engineering delegation and repository controls.
ISO/IEC 42001	AI management system and continual improvement	Provides organizational governance context; DH-ADOM is an engineering execution model, not a replacement for an AIMS.
NIST Agent Identity & Authorization work	Identity and authorization for software and AI agents	Supports explicit identity/authorization requirements; DH-ADOM applies these controls to development-agent hierarchy.
OWASP Agentic Security Initiative	Agentic threat and security guidance	Supports the security threat model; DH-ADOM incorporates security into delegation and agent lifecycle controls.

Adjacent work	Primary contribution	Relationship to DH-ADOM
BOAD / hierarchical SWE agents	Specialized sub-agent hierarchies for software engineering tasks	Shows the value of hierarchy in SWE execution; DH-ADOM adds persistent repository ownership, governance, lifecycle, and evidence semantics.

The most important publication implication is therefore precision: DH-ADOM should not be marketed as the first hierarchical agent architecture or the first AI-SDLC framework. Current public work already covers portions of those spaces. Its defensible contribution is the explicit engineering operating model that turns hierarchy into durable ownership boundaries, bounded delegation contracts, and repository-native lifecycle governance.

4. DH-ADOM Model

4.1 Core entities

DH-ADOM defines four governance layers and three execution roles.

Entity	Definition
Engineering Constitution	Repository-wide set of non-negotiable development rules and precedence.
AI-SDLC	Lifecycle model governing software intent, requirements, architecture, implementation, validation, release, and learning.
ADLC	Lifecycle model governing agent intent, contract, identity, capabilities, tools, memory, permissions, autonomy, validation, certification, deployment, and retirement.
Root Agent	Global project orchestrator responsible for decomposition, routing, review, integration, global state, and governance.
Feature Agent	Persistent owner of a bounded feature/domain and its local engineering artifacts.
Specialist Agent	Task-scoped agent that performs a bounded technical activity under a parent Feature Agent.
Delegation Contract	Machine-readable authorization and scope record connecting parent, child, task, permissions, budget, and expected output.

4.2 Hierarchy

See Figure A1 in Appendix A for the vector diagram.

4.3 Graph representation

The model can be represented as a directed labeled graph $G = (V, E)$, where V is the set of agents, features, tasks, and governance artifacts, and E contains relations such as owns, delegates_to, reviews, depends_on, handoffs_to, and validates.

```
V = Agents union Features union Tasks union Artifacts

E subset-of {
  owns,
  delegates_to,
  reviews,
  depends_on,
  handoffs_to,
  validates,
  governed_by
}

For each delegation d:
d = (parent, child, task, scope, permissions, budget, lifetime, evidence)

Core safety constraints:
depth(d) <= Dmax
parallel(d) <= Pmax
cost(d) <= Cmax
runtime(d) <= Tmax
scope(child) subset-of scope(parent) union explicitly granted scope
```

5. Design Principles

Governance before implementation. The repository must establish its engineering constitution, lifecycle model, agent hierarchy, and delegation rules before feature execution.

Ownership is explicit. Every meaningful feature/domain has an identifiable owner agent.

Delegation is bounded. Every delegated task has scope, permissions, budget, lifetime, and acceptance criteria.

Root is an orchestrator, not a super-agent. The Root Agent coordinates rather than absorbing every technical responsibility.

Agents are lifecycle-managed. Agents have intent, contracts, identity, passports, capabilities, skills, tools, permissions, autonomy, evaluation, and retirement.

Repository is the source of truth. AI agents should be able to resume work from versioned repository state without hidden chat history.

Evidence over claims. Completion is established through tests, evaluation, artifacts, and traceability.

Cross-boundary changes are controlled. Feature-owned domains cannot be modified silently by unrelated agents.

Progressive autonomy. Autonomy should be bounded and increased only through evidence and policy.

Research before invention. Novel functionality is preceded by prior-art analysis and an explicit build/buy/extend decision.

6. Engineering Constitution

The Engineering Constitution is the highest repository-level development authority below system safety. It defines what an AI agent may and may not do while working on the codebase. It should be committed to the repository, versioned, reviewable, and enforced by the Root Agent and supporting tooling.

Constitution concern	Required behavior
Precedence	Higher-level rules override lower-level prompts, tasks, and implementation choices.
No blind coding	Agents must inspect context, architecture, tests, and existing components before writing code.
No invented architecture	Unknown architectural facts become explicit assumptions or research tasks.
No silent deletion	Destructive changes require explicit justification and traceability.
No uncontrolled spawning	Agent creation must follow delegation policy.
No silent ownership crossing	Cross-feature modifications require coordination and review.
Evidence	Meaningful work must produce auditable evidence.
Resumability	Current state and handoff artifacts remain current.

6.1 Recommended root files

```
/ENGINEERING_CONSTITUTION.md
/AGENTS.md
/.ai-sdlc/
/docs/ai-sdlc/
/docs/adlc/
/agents/root/AGENT.md
/agents/templates/FEATURE_AGENT.md
/agents/templates/SPECIALIST_AGENT.md
/.ai-sdlc/agent-registry.yaml
/.ai-sdlc/delegation-policy.yaml
/.ai-sdlc/quality-gates.yaml
/.ai-sdlc/traceability.yaml
```

7. AI-SDLC and ADLC integration

DH-ADOM does not replace software lifecycle governance or agent lifecycle governance. It provides the hierarchy through which those lifecycles are executed.

See Figure A2 in Appendix A for the vector diagram.

AI-SDLC: Intent -> Requirements -> Architecture -> Design -> Task -> Implementation -> Validation -> Evaluation -> Security -> Certification -> Release -> Learning
ADLC: Agent Intent -> Contract -> Identity -> Passport -> Capabilities -> Skills -> Tools -> Memory -> Permissions -> Autonomy -> Implementation -> Testing -> Evaluation -> Security -> Promotion -> Deployment -> Continuous Evaluation -> Drift -> Revalidation -> Retirement

This approach is consistent with the growing view that AI agent development needs lifecycle-specific controls. Salesforce's ADLC guidance explicitly treats agents as systems that reason, act, and learn, while AI-SDLC initiatives are formalizing declarative governance, orchestration, autonomy policies, and auditability. [3][4]

8. Root Agent

The Root Agent is responsible for project-level coordination. It interprets requirements, identifies the affected domain, selects the appropriate Feature Agent, authorizes bounded delegation, reviews returned evidence, and updates global project state. Its most important design property is restraint.

Root capability	Expected behavior	Failure to prevent
Requirement interpretation	Convert user intent into a governed task.	Directly coding from ambiguous prompts.
Feature routing	Identify the authoritative Feature Agent.	Duplicate ownership.

Root capability	Expected behavior	Failure to prevent
Delegation	Create bounded delegation contracts.	Unbounded agent spawning.
Review	Validate output against requirements and evidence.	Blind trust in child agents.
Integration	Coordinate cross-feature change.	Silent cross-boundary modifications.
State	Maintain global project state.	Stale or contradictory project context.

9. Feature Agents

Feature Agents are persistent owners of bounded engineering domains. They are not merely prompts stored next to source code. Their purpose is to establish an explicit ownership boundary for requirements, design, implementation, tests, evaluation, evidence, risks, and handoff.

```
features/  
  identity/  
    AGENT.md  
  evaluation/  
    AGENT.md  
  simulation/  
    AGENT.md  
  security/  
    AGENT.md  
  certification/  
    AGENT.md
```

10. Specialist Agents

Specialists are created only when a task requires specialized expertise, independent validation, risk isolation, or useful parallelism. A Specialist normally has task-scoped lifetime and should return a structured handoff to its parent.

Delegation parameter	Why it matters
Parent	Establishes authority and accountability.
Purpose	Prevents generic or unexplained agent creation.
Scope	Limits actions to the delegated problem.

Delegation parameter	Why it matters
Permissions	Applies least privilege.
Budget	Controls cost and runtime.
Lifetime	Prevents abandoned agents from remaining active.
Depth	Prevents recursive agent explosion.
Evidence	Makes work auditable.
Acceptance criteria	Defines what constitutes completion.

11. Delegation Governance

Delegation is a first-class engineering action. DH-ADOM treats a delegation as a contract, not an implicit side effect of a prompt.

```

DelegationContract:
  delegation_id
  parent_agent
  child_agent
  task
  scope
  inputs
  expected_outputs
  permissions
  tools
  risk
  autonomy
  max_steps
  max_runtime
  max_cost
  max_tool_calls
  max_parallel_agents
  max_delegation_depth
  termination_condition
  evidence_requirements
  acceptance_criteria

```

A child agent's effective authority should be the intersection of its inherent authority and the authority explicitly delegated by its parent. Any request outside that scope must be rejected or escalated.

12. Repository-Native Implementation Model

The repository is the durable memory of the development organization. A new AI model should be able to understand the product, current state, decisions, ownership boundaries, active work, and next actions without access to prior conversation history.

Repository artifact	Purpose
PROJECT_CONTEXT.md	Product context and operating assumptions.
CURRENT_STATE.md	What actually exists.
CURRENT_STATE_US.md	Operational status and blockers.
NEXT_ACTIONS.md	Immediate executable work.
AI_HANDOFF.md	Continuation context between agents.
AGENTS.md	Repository-level operating instructions.
Feature AGENT.md	Local feature ownership and behavior contract.
.ai-sdlc/*	Machine-readable lifecycle and policy state.
Agent Registry	System of record for agent identity and ownership.

13. Security and Trust Controls

DH-ADOM treats agent identity, authorization, delegation, and autonomy as security boundaries. This is aligned with current standards work: NIST is explicitly studying standards-based identity and authorization for software and AI agents, noting that autonomous action at scale creates new risks; OWASP's 2026 Agentic Applications Top 10 likewise frames agentic systems as a distinct security problem. [6][7]

Control	DH-ADOM interpretation
Agent identity	Agent identity is independent of model identity.

Control	DH-ADOM interpretation
Least privilege	Feature and Specialist Agents receive only required permissions.
Side-effect control	Consequential actions pass through policy and authorization.
Autonomy budget	Steps, cost, time, tools, and delegation depth can be capped.
Kill switch	Execution can be stopped, isolated, revoked, or quarantined.
Audit	Delegation and modification events are recorded.
Cross-feature isolation	Agents cannot silently modify unrelated domains.
Evidence integrity	Lifecycle evidence is retained and traceable.

13.1 Relationship to external governance

DH-ADOM should be implemented alongside, not instead of, organizational AI governance. ISO/IEC 42001 defines an AI management system for establishing, implementing, maintaining, and continually improving AI governance. NIST AI RMF and its Generative AI Profile provide risk-management guidance, while OWASP provides agentic security guidance. DH-ADOM maps engineering execution and delegation into that larger governance ecosystem. [5][8][9]

14. Evidence, Traceability, and Auditability

DH-ADOM treats evidence as a first-class artifact. A completed task should be reconstructable from requirement through implementation and validation.

See Figure A7 in Appendix A for the vector diagram.

Intent -> Requirement -> Feature -> Feature Agent -> Task -> Specialist Delegation (if any) -> Implementation -> Tests -> Evaluation -> Evidence -> Feature Handoff -> Root Review -> Integration -> Release -> Production Outcome

This lineage supports both operational debugging and organizational accountability. It also aligns with the broader direction of declarative AI-SDLC governance toward quality gates, auditable execution, and machine-readable lifecycle state. [4]

15. Evaluation Framework

DH-ADOM should not be evaluated only on whether agents can produce code. It should be evaluated on whether the development organization behaves predictably under normal and adversarial conditions.

Dimension	Representative measurements
Routing accuracy	Correct Feature Agent selected for a task.
Ownership integrity	Unauthorized cross-feature modifications blocked.
Delegation efficiency	Useful specialists created only when justified.
Boundedness	Budget and depth constraints enforced.
Handoff quality	Independent continuation from repository state succeeds.
Traceability	Requirement-to-release lineage is reconstructable.
Recovery	Failed agents terminate or recover safely.
Security	Privilege escalation and governance bypass are blocked.
Cost	Delegation and execution remain within policy.
Maintainability	Agent responsibilities remain discoverable and bounded.

15.1 Suggested experimental design

Construct a representative software-engineering task corpus covering isolated feature work, cross-domain changes, maintenance, security changes, refactoring, and release tasks.

Run identical tasks under a monolithic-agent baseline and under DH-ADOM.

Measure routing accuracy, unauthorized changes, agent count, delegation depth, execution cost, completion quality, and recovery behavior.

Inject failures such as specialist timeout, stale state, incorrect feature routing, permission denial, and recursive spawning pressure.

Compare the ability of a fresh AI session to resume work from repository artifacts.

Publish the experiment scripts, task set, environment definition, and raw results with the white paper version.

No empirical superiority claim should be made until this type of controlled evaluation has been executed and published.

16. Reference Architecture for DH-ADOM Adoption

See Figure A8 in Appendix A for the vector diagram.

The physical architecture can still use services, workers, queues, databases, and cloud infrastructure as appropriate. DH-ADOM is primarily an organizational and control-plane model; it does not require a microservice boundary for every feature.

17. Adoption Guide

Phase 0 - Governance bootstrap

- Create the Engineering Constitution and AGENTS.md.
- Create AI-SDLC and ADLC machine-readable lifecycle artifacts.
- Create Root Agent specification and registry.
- Create Feature Agent and Specialist Agent templates.
- Create delegation policy and quality gates.

Phase 1 - Root and ownership

- Implement Root Agent routing and global state.
- Register meaningful product domains.
- Assign Feature Agents and explicit ownership.
- Implement cross-feature review and escalation.

Phase 2 - Delegation controls

- Implement Delegation Contracts.
- Enforce permissions and autonomy budgets.
- Add specialist creation/termination rules.
- Add audit and traceability.

Phase 3 - Evidence and resumability

- Standardize state and handoff artifacts.
- Validate fresh-session continuation.
- Link tasks to requirements and evidence.

Phase 4 - Continuous assurance

- Feed incidents and production outcomes back into AI-SDLC.
- Measure drift in ownership, contracts, permissions, and architecture.
- Periodically re-certify agent roles and delegation policies.

18. Limitations and Non-Claims

- DH-ADOM is an operating model, not a replacement for an agent framework, model runtime, protocol, or IAM system.
- DH-ADOM does not guarantee that agents are safe merely because a hierarchy exists.
- Hierarchy can increase coordination overhead; not every task requires a Feature Agent plus Specialist Agent.
- Root/Feature/Specialist roles must be implemented with explicit policies to avoid role formalism without real enforcement.
- The model requires empirical evaluation to establish productivity, reliability, cost, or quality improvements.
- This paper does not claim that hierarchical multi-agent orchestration, AI-SDLC, ADLC, or repository agent instructions were invented by DevHeal Labs AI.
- The contribution is the specific composition and formalization of repository-level hierarchy, ownership, bounded delegation, lifecycle integration, and evidence-oriented handoff represented by DH-ADOM.

19. Research Positioning and Contribution Statement

Based on the literature and public engineering material reviewed for this paper, the individual ingredients of DH-ADOM have substantial precedent. Orchestrator-worker delegation is an established agent pattern; AutoGen and related frameworks provide mechanisms for multi-agent composition; hierarchical specialized agents have been studied for software engineering; ADLC and AI-SDLC frameworks are emerging; and identity, authorization, risk, and governance are active standards efforts. [1][2][3][4][6][10]

The proposed research contribution of DH-ADOM is therefore best stated as a system-level synthesis: a repository-native operating model in which project-wide orchestration, persistent feature ownership, task-scoped specialist delegation, explicit delegation contracts, bounded autonomy, AI-SDLC/ADLC enforcement, state/handoff continuity, and traceable evidence are treated as one coherent engineering organization.

A future peer-reviewed version should strengthen this contribution with empirical comparisons against monolithic agents, flat multi-agent teams, conventional orchestrator-worker patterns, and existing

AI-SDLC implementations.

20. Open Research Questions

1. What is the optimal granularity for Feature Agents?
2. When does Feature-Agent permanence outweigh the cost of maintaining a larger agent registry?
3. How should delegation depth and parallelism be optimized against quality and cost?
4. Can agent ownership be inferred automatically from repository structure and dependency graphs?
5. How should Root Agents detect that a Feature Agent has accumulated too much responsibility?
6. How should delegation contracts be verified automatically from runtime traces?
7. Can agent autonomy be promoted using statistical confidence and sustained evidence?
8. How should hierarchical agent systems handle conflicting local objectives?
9. What metrics best measure handoff quality between independent AI sessions?
10. How should agent role drift be detected over long-lived repositories?
11. Can DH-ADOM become interoperable with MCP, A2A, AGENTS.md and declarative AI-SDLC specifications without coupling to any single vendor?

21. Publication Package for DevHeal Labs AI

For public release, the strongest approach is not to publish only a PDF. Publish a versioned evidence package so readers can distinguish the concept, specification, implementation, and empirical results.

Artifact	Publish?	Purpose
DH-ADOM White Paper PDF	Yes	Canonical technical narrative.
DH-ADOM Specification	Yes	Normative rules, schemas, lifecycle, delegation semantics.
Reference templates	Yes	AGENTS.md, Root Agent, Feature Agent, Specialist Agent, delegation contract.
Reference implementation	Yes when stable	Demonstrates that the model is executable.
Conformance / validation suite	Strongly recommended	Shows whether an implementation actually follows DH-ADOM.

Artifact	Publish?	Purpose
Benchmark / experiment package	Strongly recommended	Supports empirical claims.
Architecture diagrams	Yes	Improves adoption and comprehension.
Changelog / version history	Yes	Makes the model auditable and citable.
DOI archive	Recommended	Persistent versioned citation and preservation.

Zenodo is useful for the archival layer because published records receive a DOI and can preserve software, documentation, and related research artifacts. A versioned white paper can therefore be cited independently of the live website. [11]

22. Website Publication Structure

devheallabs.com/dh-adom/

Hero

DH-ADOM

DevHeal Hierarchical Agent Development & Orchestration Model

"A governed operating model for AI-native software engineering."

Problem

Why multi-agent development needs explicit ownership and delegation.

Model

Constitution -> AI-SDLC -> ADLC -> Root -> Feature -> Specialist.

Architecture

Interactive hierarchy diagram.

Principles

Ownership, bounded delegation, evidence, resumability, governance.

Specification

Download / browse normative specification.

White Paper

Download PDF + DOCX.

Implementation

Reference repository / examples.

Conformance

Validation suite + results.

Research

Benchmarks, experiments, papers, revisions.

Standards Mapping

NIST / ISO / OWASP / MCP / A2A / AI-SDLC.

Versions

v1.0, v1.1, ...

Citation

DOI + BibTeX + citation text.

23. Standards and Ecosystem Mapping

External ecosystem	Scope	DH-ADOM role
NIST AI RMF	AI risk management	Engineering governance and evidence can map into risk-management activities.
ISO/IEC 42001	AI management system	DH-ADOM provides engineering execution controls within a broader AIMS.
OWASP Agentic Applications	Agentic security risk	Threats become security validation inputs for agent hierarchy and delegation.
MCP	Agent/tool connectivity	DH-ADOM governs how repository agents may use MCP-connected tools.
A2A	Agent-to-agent interoperability	DH-ADOM can use A2A for inter-agent communication while preserving parent/child governance.
AGENTS.md	Repository agent instructions	DH-ADOM can use AGENTS.md as a project-level instruction surface; deeper ownership lives in Feature Agent artifacts.
AI-SDLC frameworks	AI-assisted software lifecycle governance	DH-ADOM can operate as a hierarchical organizational model within or alongside an AI-SDLC implementation.

24. Conclusion

AI-native software engineering is not simply traditional development with an LLM added to the editor. As agents gain the ability to reason, use tools, modify repositories, and coordinate other agents, the development organization itself becomes partially executable.

DH-ADOM proposes that this organization should be explicit. A Constitution defines the non-negotiable rules. AI-SDLC defines software lifecycle governance. ADLC defines agent lifecycle governance. A Root Agent coordinates the project. Feature Agents own bounded domains. Specialist Agents perform narrowly scoped work under delegation contracts. Evidence and handoff artifacts make the organization observable and resumable.

The central principle is simple:

THE ROOT AGENT ORCHESTRATES. THE FEATURE AGENT OWNS. THE SPECIALIST AGENT ASSISTS. AI-SDLC GOVERNS DEVELOPMENT. ADLC GOVERNS AGENTS. EVIDENCE PROVES COMPLETION.

The next step is empirical: implement the specification as a reference system, publish a conformance suite, and measure whether explicit hierarchical ownership and bounded delegation improve reliability, cost, security, resumability, and maintenance compared with less structured agent organizations.

References

- [1] Anthropic. "Building effective agents." 19 Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- [2] Wu, Q. et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." COLM 2024. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/>
- [3] Salesforce Architects. "The Agent Development Lifecycle: From Conception to Production." <https://architect.salesforce.com/docs/architect/fundamentals/guide/agent-development-lifecycle>
- [4] AI-SDLC Framework. "AI-SDLC Framework Primer" and public specification. <https://ai-sdlc.io/docs/spec/primer>
- [5] ISO. "ISO/IEC 42001:2023 - Artificial intelligence - Management system." <https://www.iso.org/standard/42001>
- [6] NIST NCCoE. "Software and AI Agent Identity and Authorization." <https://www.nccoe.nist.gov/projects/software-and-ai-agent-identity-and-authorization>
- [7] OWASP GenAI Security Project. "OWASP Top 10 for Agentic Applications for 2026." <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- [8] NIST. Autio, C. et al. "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile." NIST AI 600-1, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [9] NIST. "Artificial Intelligence Risk Management Framework (AI RMF)." <https://www.nist.gov/itl/ai-risk-management-framework>
- [10] Xu, I. et al. "BOAD: Discovering Hierarchical Software Engineering Agents via Bandit Optimization." arXiv:2512.23631, 2025. <https://arxiv.org/abs/2512.23631>
- [11] Zenodo. "Quick start" and DOI guidance for research objects. <https://help.zenodo.org/docs/get-started/quickstart/>
- [12] Google Developers Blog. "Announcing the Agent2Agent Protocol (A2A)." 9 Apr 2025. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>

Research Note

The related-work section was updated in October 2026 using public material from Anthropic, Microsoft Research, Salesforce Architects, the AI-SDLC Framework project, NIST, ISO, OWASP, Google, and recent academic work on hierarchical software-engineering agents. The field is fast-moving; publication versions of this paper should retain a dated bibliography and update the comparison table in subsequent revisions.

Appendix A - DH-ADOM Architecture Figures

Figure A1. DH-ADOM hierarchy.

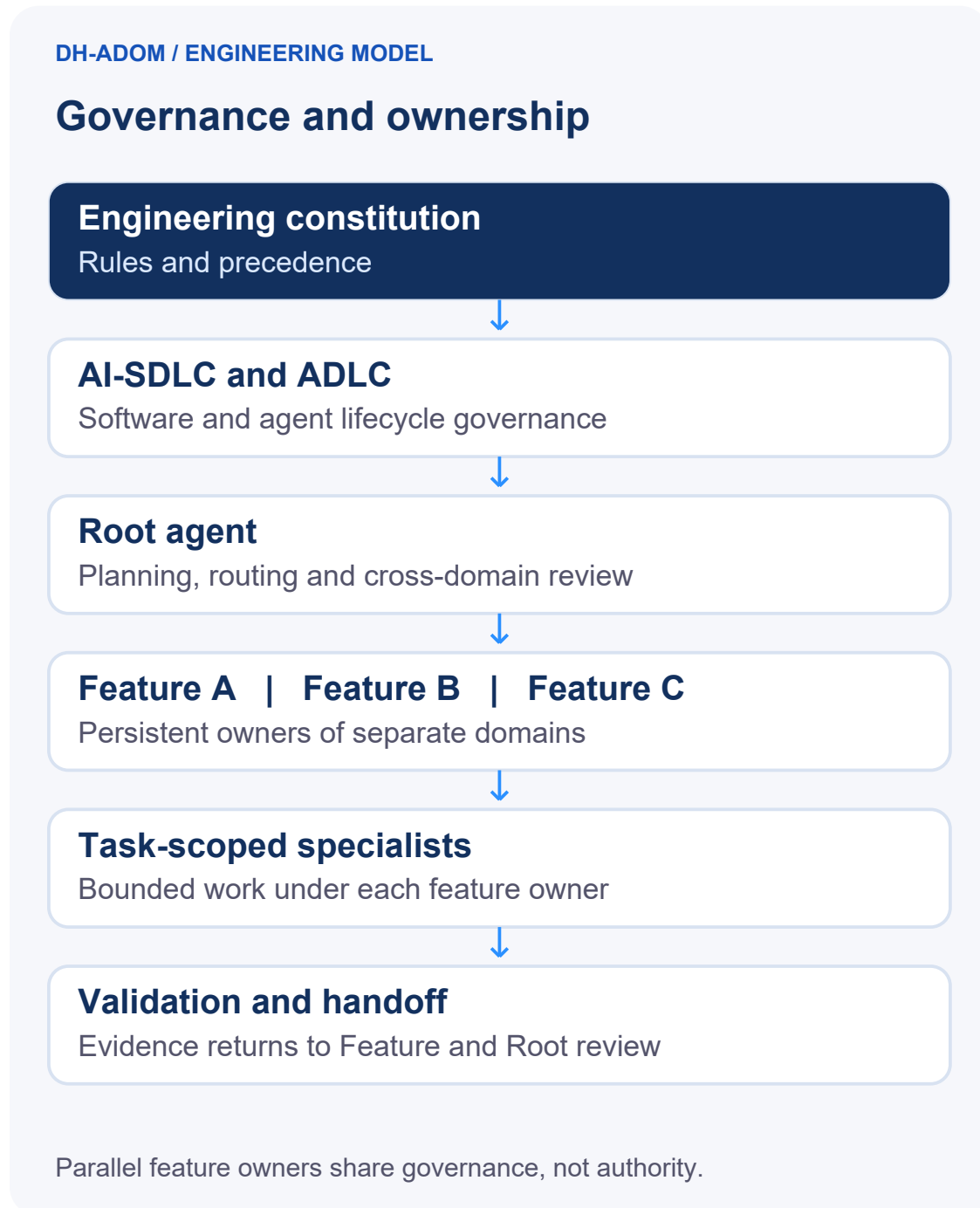


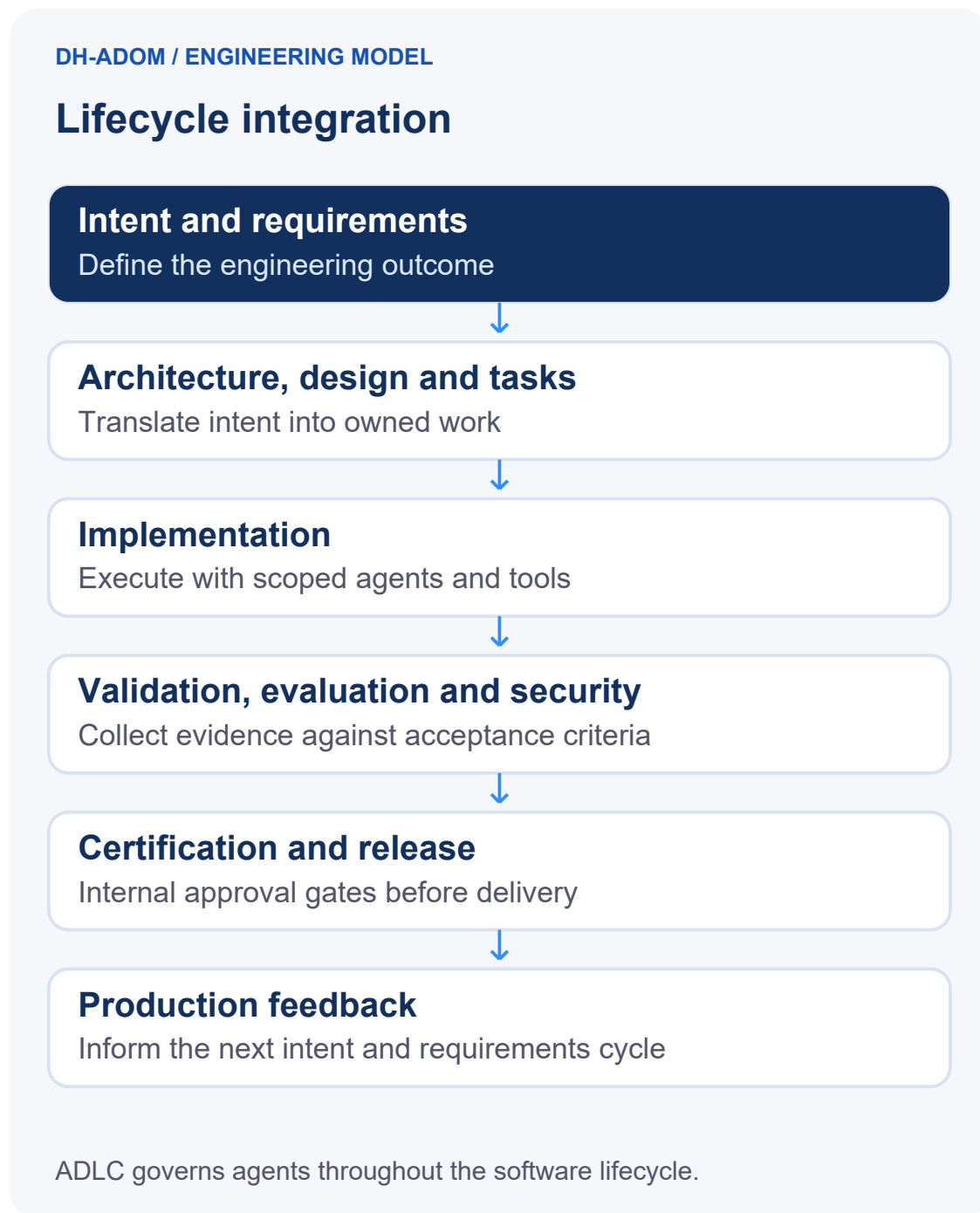
Figure A2. Lifecycle integration.

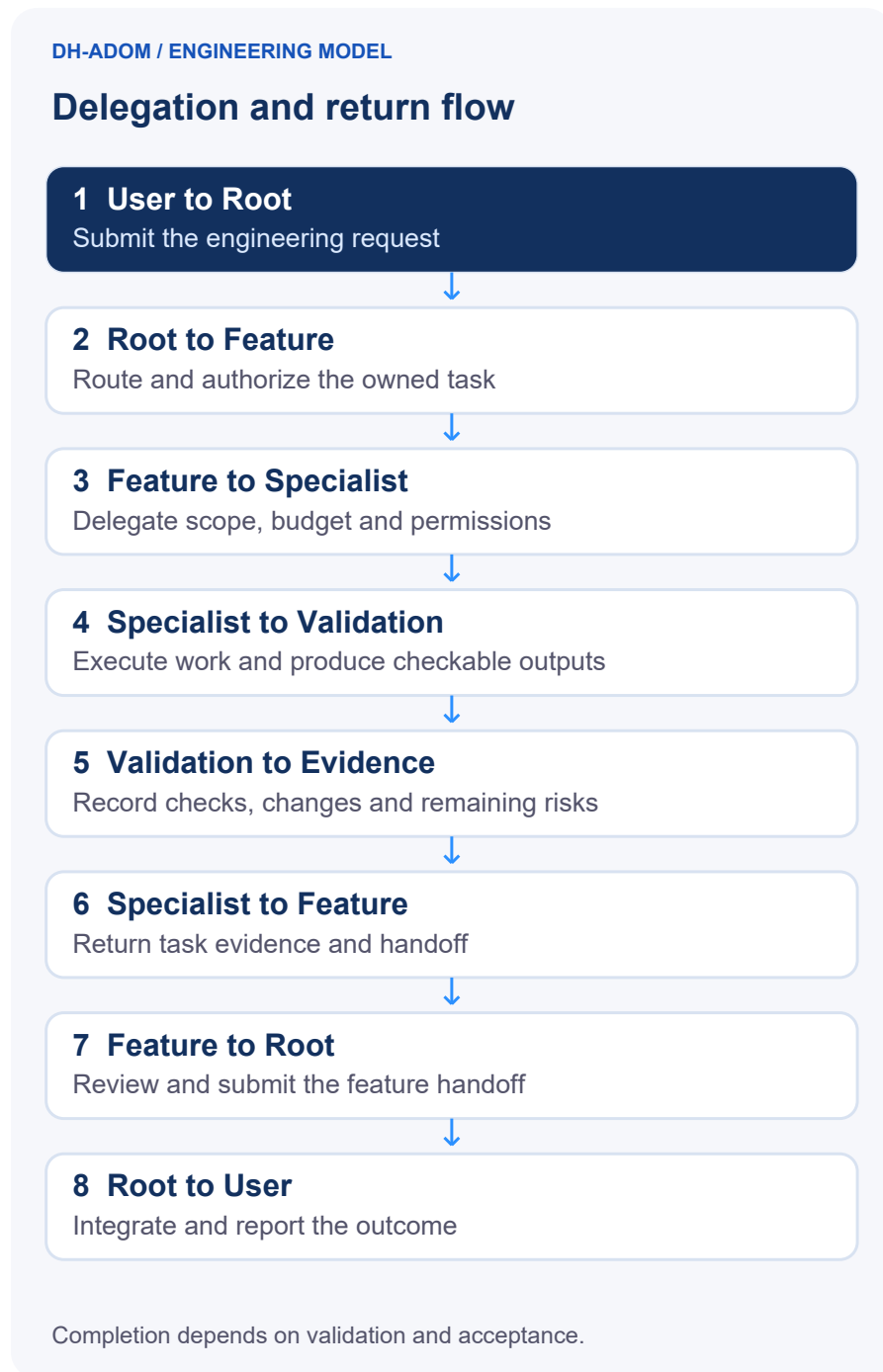
Figure A3. Delegation sequence.

Figure A4. Agent lifecycle state machine.**DH-ADOM / ENGINEERING MODEL****Agent lifecycle states****PROPOSED to DESIGNED**

Define intent, ownership and contract

DESIGNED to REGISTERED

Record a distinct agent identity

REGISTERED to VALIDATED

Evaluate capabilities and constraints

VALIDATED to CERTIFIED to ACTIVE

Internal approval followed by activation

ACTIVE to SUSPENDED

Pause execution for review

SUSPENDED to REVALIDATION to ACTIVE

Recheck the agent before resuming

ACTIVE to DEPRECATED to RETIRED

Withdraw and retire the agent

SUSPENDED to RETIRED

Terminate without resuming execution

Transitions follow the supplied agent lifecycle model.

Figure A5. Feature ownership boundaries.**DH-ADOM / ENGINEERING MODEL****Feature ownership boundaries****Root coordinator**

Routes and coordinates work across domains

Feature A owns features/a/**

Changes remain within the declared boundary

Feature B owns features/b/**

A separate owner governs this domain

Cross-feature change

Coordinate through Root and affected owners

Review and traceability

Record approvals, changes and handoffs

Feature A and B are peers, not a delegation chain.

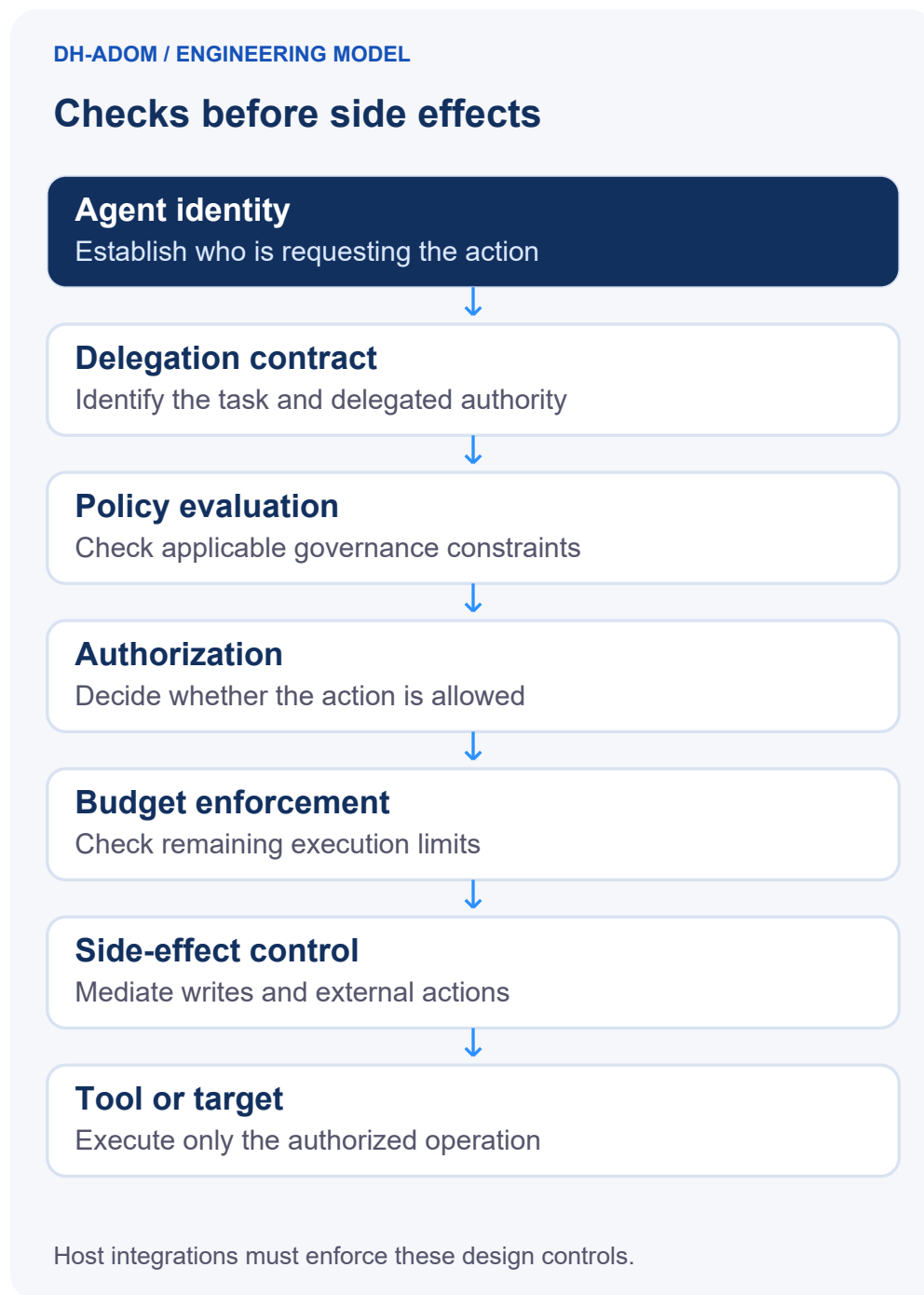
Figure A6. Security and delegation controls.

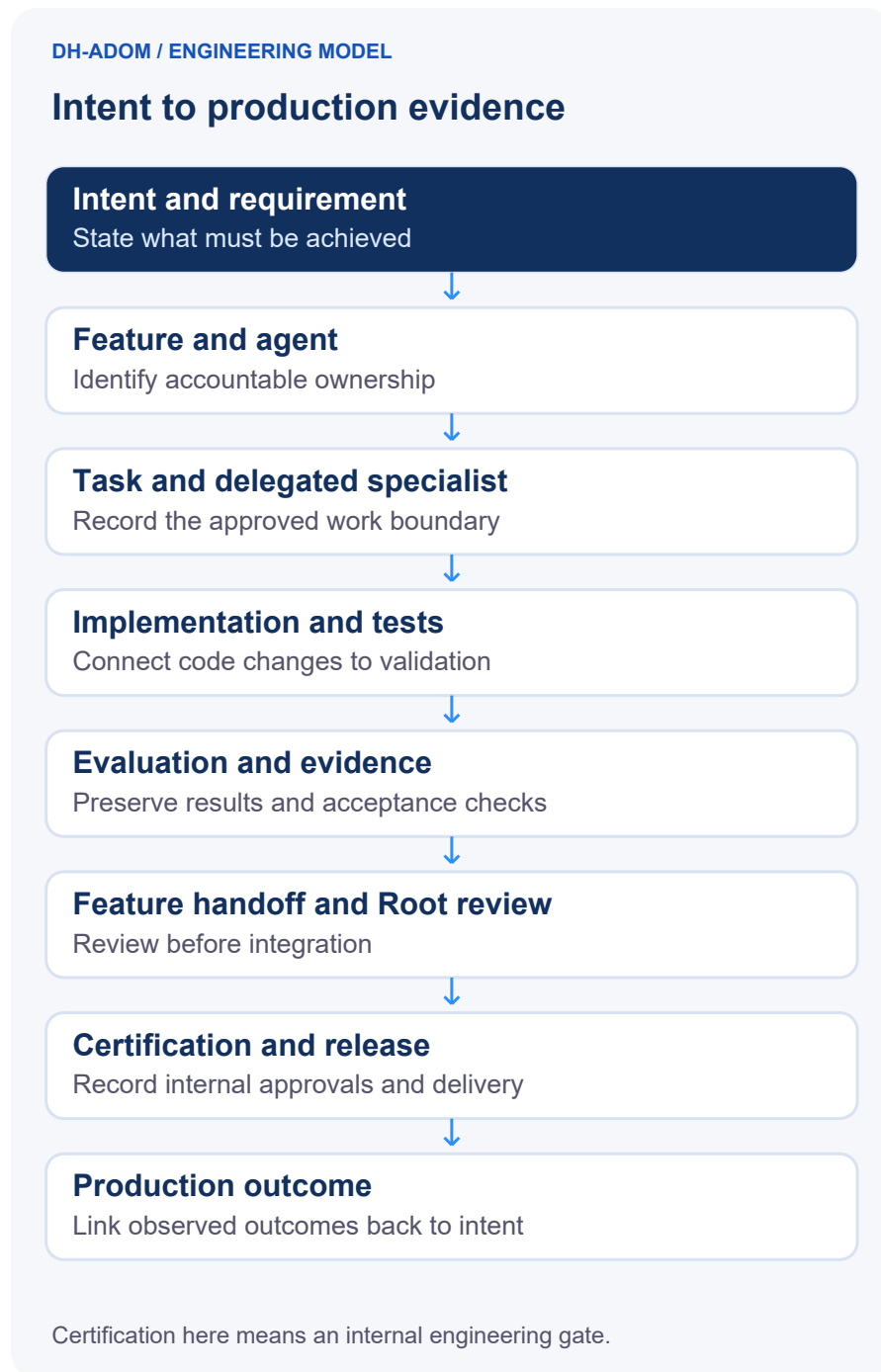
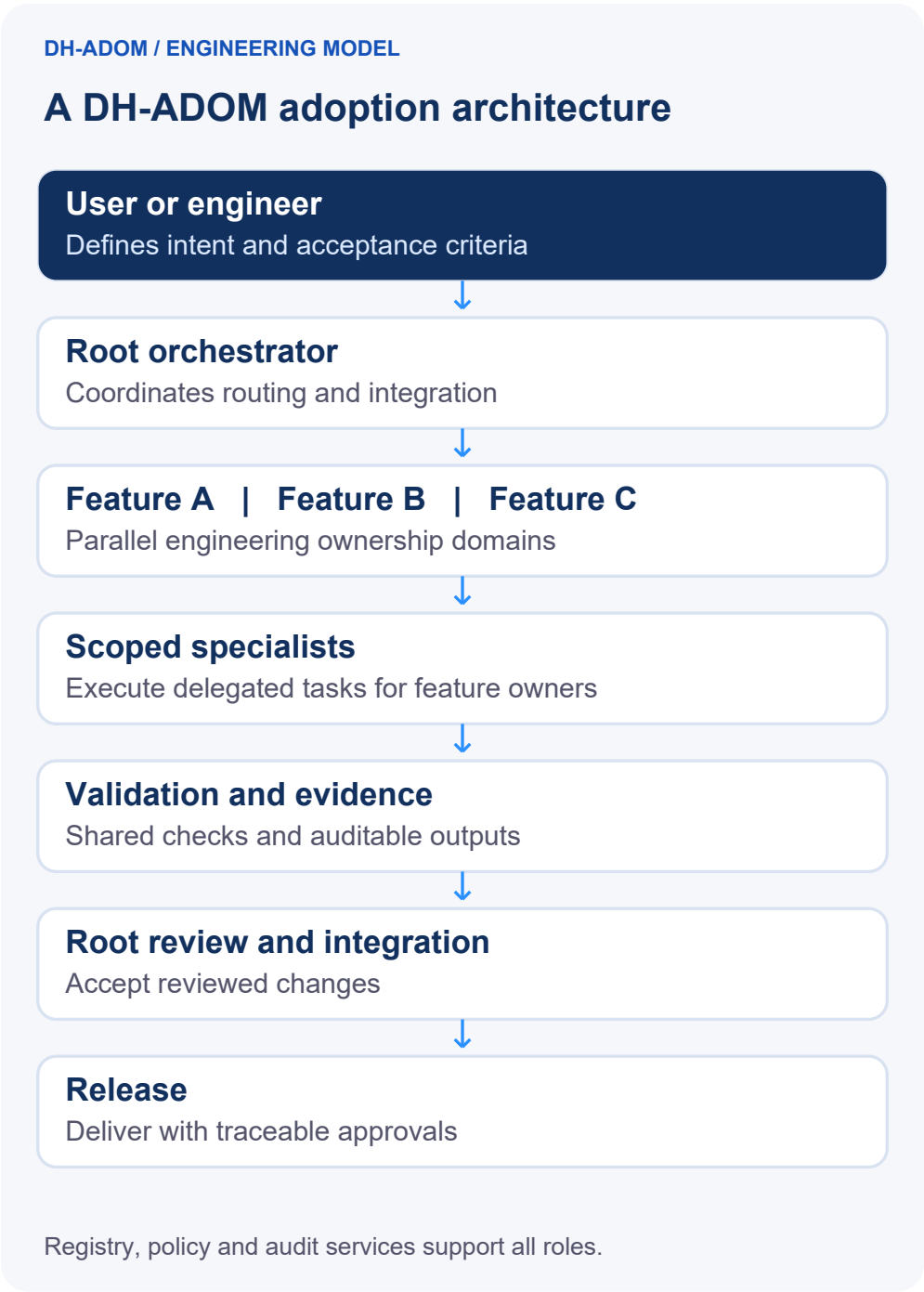
Figure A7. Requirement-to-production traceability.

Figure A8. Reference architecture.



Appendix B - Publication Package

The DH-ADOM publication package 1.1.0 is distributed as a versioned publication and implementation package containing the white paper, normative technical specification, reference templates, reference implementation, conformance suite, benchmark definitions, architecture diagrams, research materials, website-ready content, and citation/versioning assets.